

UE03 · DONNÉES ET ANALYTICS

Observabilité et détection de comportements anormaux sur un jeu multijoueur web

Imrane Mesbahi

Octobre 2026

github.com/Maoni21/game-observability

Sommaire

1. Introduction
2. Qualité des données (B3)
3. Métriques ou logs (C3)
4. Typologie des rapports et comportements anormaux (E1, E2)
5. Alertes (E3)
6. Postmortem (F)
7. Conclusion

1. Introduction

Le sujet porte sur un jeu de tir multijoueur qui tourne dans le navigateur. Le client envoie un rapport de performance au serveur dès qu'une frame met trop de temps à s'afficher, et le serveur journalise chaque fin de partie. Au départ, rien ne permettait d'exploiter ces données en dehors d'un export manuel. La direction posait deux questions : pourquoi des joueurs se plaignent de saccades depuis jeudi soir, et est-ce que quelqu'un abuse du jeu ?

J'ai mis en place une stack d'observabilité qui démarre avec un seul `docker compose up`. Prometheus scrape les métriques exposées par le service sur `/metrics`. Alloy collecte les logs du service ainsi que l'export de la semaine passée, et les pousse dans Loki. Grafana affiche l'ensemble dans trois dashboards provisionnés. Les alertes sont réparties entre Prometheus pour la santé du service et le ruler de Loki pour le comportement des joueurs. Une CI GitHub Actions lance le lint et les tests, construit l'image Docker, la scanne avec Trivy puis la publie sur GHCR.

Les réponses aux deux questions, avant le détail :

Saccades. La cause est le build `beta-20260924-3`, déployé le jeudi 24/09 vers 19 h. Le rendu de l'overlay y prend entre 260 et 690 ms par frame, mais uniquement sur les grands écrans (2560 px et plus) avec le bloom activé, configuration qui représente 95 % des rapports overlay. Les deux builds suivants n'ont pas corrigé le problème.

Abus. Il y en a deux. Du farming de victoires : 83 parties gagnées 3-0 sur la carte vault en moins de 3 minutes, sur deux nuits, dont 17 seulement ont été mises en quarantaine par l'anti-triche. Et un client, `5e1f0c7a`, qui envoie des rapports falsifiés (70 au total).

2. Qualité des données (B3)

C'est la partie qui m'a demandé le plus de temps, car l'export était loin d'être exploitable tel quel. Le fichier fait 264 344 lignes pour 2 944 blocs (2 148 rapports et 796 parties). Le script `scripts/parse-export.js` le normalise avant l'ingestion dans Loki. Après traitement, il reste 2 889 événements (2 110 rapports et 779 parties). Je retrouve exactement ce total en comptant dans Loki, donc rien n'a été perdu à l'ingestion.

Les problèmes rencontrés, dans l'ordre où je les ai découverts :

Problème	Volume	Traitement
Événements sur plusieurs lignes (titre, résumé, puis JSON indenté)	2 944 blocs	Découpage par bloc, une ligne JSON par événement
Dates au format américain sur 12 h, où <code>12:05 AM</code> correspond à minuit et non à midi	100 dates	Conversion en ISO 8601, avec un test unitaire dédié au cas 12 AM
Heures en UTC+2 sans aucune indication de fuseau	2 h de décalage	Décalage déduit en comparant avec la date de création des serveurs

Problème	Volume	Traitement
Fichier en ordre antéchronologique, avec en plus 2 blocs mal placés	2 blocs	Tri chronologique complet
Événements en double	55	Dédoublonnage (2 944 - 55 = 2 889)
Changement de schéma sur les 25 derniers rapports : rttMs a changé de place	25 rapports	Repéré à cause d'un panneau vide dans Grafana ; les dashboards lisent maintenant les deux versions
Aucune donnée le 22/09 entre 11:59 et 14:14 UTC	2 h 15	Non récupérable, signalé. Sur le même créneau, la veille et le lendemain comptent 18 et 38 événements
Rapports physiquement impossibles (temps négatifs)	70	Conservés mais marqués <code>falsifie</code> , et exclus des statistiques

Pour chaque chiffre, la commande ou la requête qui permet de le retrouver est dans `docs/qualite.md`. Par exemple, le contrôle du total dans Loki :

```
sum by (event) (count_over_time({service="game-telemetry", origin="export"} [$_range]))
→ perf_spike : 2110, game_completed : 779
```

3. Métriques ou logs (C3)

Pour décider où stocker une information, je suis parti d'une question simple : est-ce que je veux savoir **combien**, ou bien **lequel** et **pourquoi** ? « Combien » relève d'une métrique Prometheus : une valeur agrégée, peu coûteuse à stocker et adaptée au déclenchement d'alertes. « Lequel » et « pourquoi » relèvent d'un log dans Loki, qui conserve le détail complet de l'événement.

Information	Stockage
Nombre de requêtes, taux d'erreurs, latence (RED)	Prometheus
Parties en cours et terminées	Prometheus
Rapports par cause et par build	Les deux : la tendance dans Prometheus, le détail dans Loki
Écran, navigateur, joueur	Loki, la cardinalité serait beaucoup trop élevée pour Prometheus
Export de la semaine passée	Loki, qui accepte l'ingestion d'événements horodatés dans le passé
Farming et triche	Loki, la détection a besoin du score, de la carte, des fps...

Le principal risque côté Prometheus est la cardinalité. Chaque combinaison de valeurs de labels crée une nouvelle série temporelle, donc un label à valeurs libres peut faire exploser le nombre de séries. Je n'utilise que des labels à faible cardinalité : `route`, `statut`, `cause`, `build` et `carte`. Comme la cause et le build sont envoyés par le client, ils sont validés côté serveur : une valeur qui ne respecte pas le format attendu est remplacée par `invalid` ou `other`. Sans cette validation, n'importe quel client pourrait créer des milliers de séries. Côté Loki, je me limite à 4 labels (`service`, `origin`, `level`, `event`) et j'extrais les autres champs au moment de la requête avec `| json`.

L'enquête a suivi ce découpage : la métrique a montré une hausse des rapports sur un build précis, et ce sont les logs qui ont permis d'identifier l'overlay et les écrans concernés.

4. Typologie des rapports et comportements anormaux (E1, E2)

Avant de chercher des anomalies, j'ai examiné les rapports pour les regrouper par famille, et j'en ai identifié 7. La fonction `src/classify.js` attribue automatiquement une famille à chaque rapport, qu'il vienne de l'export ou du flux en direct. Elle est couverte par des tests unitaires.

Famille	Critère	Rapports
falsifie	plus de 240 fps, ou un temps négatif	70
onglet-cache	l'onglet est passé en arrière-plan et le navigateur a suspendu le jeu : ce n'est pas une vraie saccade	158
reseau	problème de connexion, RTT élevé	92
shader	compilation des shaders en début de partie	459
overlay	rendu de l'overlay au-delà de 100 ms	262
monde	simulation du monde au-delà de 50 ms	20
rendu	tous les autres cas (rendu classique)	1 049

L'anomalie la plus nette est la régression du build `beta-20260924-3`. Avant lui, l'overlay n'expliquait que 9 rapports au total ; après, on passe à 253, et la part de l'overlay grimpe de moins de 1 % à 29 %. Les builds suivants ne corrigent rien : 34,5 % sur `0926-5`, encore 16 % sur `0926-6`. Les écrans concernés sont presque toujours les mêmes, 2560, 3440 ou 3840 px de large avec le bloom activé : 249 rapports sur 262, envoyés par 44 joueurs différents.

Le farming se repère à sa régularité : il revient deux nuits aux mêmes heures, à deux jours d'intervalle. Quant aux rapports falsifiés, ils viennent tous d'un seul client, avec un profil qui ne change jamais (Safari sur macOS, écran de 1366 px).

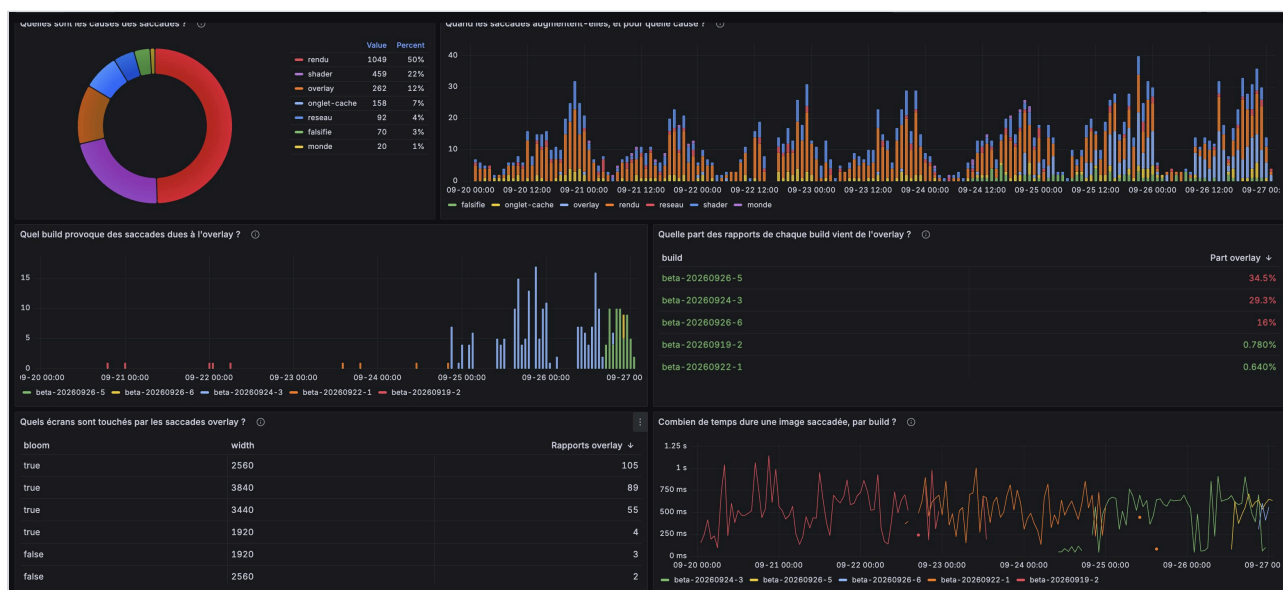


Figure 1: le dashboard « Performance côté joueur » sur la semaine de l'export.

5. Alertes (E3)

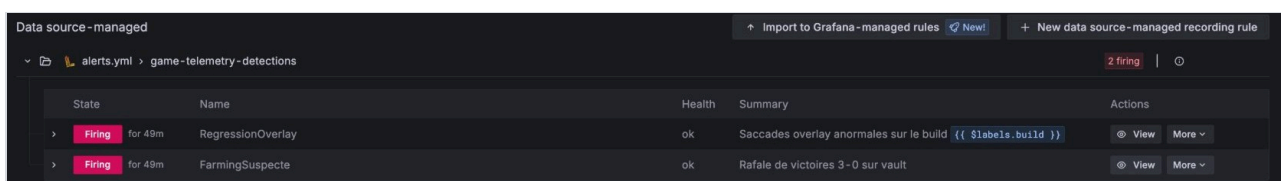
J'ai défini 7 alertes. Les seuils ne sont pas choisis au hasard : ils sont calés sur ce que j'ai observé dans les données.

Alerte	Condition	Justification du seuil
ServiceIndisponible	la cible ne répond plus au scrape pendant 1 min	soit 4 échecs consécutifs ; un échec isolé ne suffit pas
TauxErreursApiEleve	plus de 5 % d'erreurs 5xx pendant 5 min	en fonctionnement normal, il n'y en a aucune
LatenceApiElevee	p95 au-dessus de 100 ms pendant 10 min	le p95 normal tourne autour de 9 ms
BuildsInvalidesRecus	un rapport arrive avec un build au format invalide	un client légitime envoie toujours un build valide
RegressionOverlay	plus de 10 % des rapports d'un build viennent de l'overlay (fenêtre de 15 min)	moins de 1 % avant la régression, 29 % après
FarmingSuspecte	plus de 25 % de 3-0 sur vault en 1 h, avec au moins 6 parties	environ 3 % en temps normal ; le minimum de 6 parties évite les faux positifs aux heures creuses
RapportsFalsifies	au moins 3 rapports impossibles pour un même client en 1 h	un rapport isolé peut venir d'un bug, pas trois

Ces règles ont des limites connues. La plus fragile est RegressionOverlay. Sur un build qui envoie peu de rapports, le ratio est instable : 1 rapport overlay sur 3 suffit à dépasser 33 %, d'où un risque de faux positif. À l'inverse, une régression qui ne touche qu'une petite minorité de joueurs peut rester sous les 10 %. Ajouter un nombre minimum de rapports par build, comme pour FarmingSuspecte, serait une amélioration simple.

FarmingSuspecte peut aussi se déclencher sur une équipe forte qui enchaîne les 3-0, et un tricheur qui change de carte ou concède un point passe sous le radar. RapportsFalsifies ne détecte pas un client qui envoie des valeurs plausibles. Pour les alertes du service, un redémarrage volontaire déclenche ServiceIndisponible, et si Prometheus lui-même tombe, aucune alerte ne part.

Pour les valider, je les ai laissées tourner sur le flux en direct. RegressionOverlay et FarmingSuspecte se sont déclenchées comme prévu, et RapportsFalsifies est restée silencieuse, ce qui est cohérent puisque le flux en direct ne contient pas de rapports falsifiés.



The screenshot shows the Grafana alerting interface. At the top, there's a header with 'Data source - managed' and buttons for 'Import to Grafana - managed rules' and 'New!'. Below this, a breadcrumb trail shows 'alerts.yml > game-telemetry-detections'. On the right, a status bar indicates '2 firing' with a refresh icon. The main table lists two alerts:

State	Name	Health	Summary	Actions
Firing	for 49m RegressionOverlay	ok	Saccades overlay anormales sur le build {{ \$labels.build }}	View More
Firing	for 49m FarmingSuspecte	ok	Rafale de victoires 3-0 sur vault	View More

Figure 2 : RegressionOverlay et FarmingSuspecte déclenchées sur le flux en direct.

6. Postmortem (F)

Les heures sont données en heure de Paris (UTC+2), celle des joueurs.

Incident 1 : les saccades

Date	Événement
mardi 22/09, 12:10	Sortie de <code>beta-20260922-1</code> , rien d'anormal
jeudi 24/09, vers 19:00	Déploiement de <code>beta-20260924-3</code>
jeudi 24/09, 20:16	Premier rapport de saccade causé par l'overlay
vendredi 25/09	La majorité des joueurs est passée sur le nouveau build : 114 rapports overlay dans la journée
samedi 26/09, 12:32	Sortie de <code>beta-20260926-5</code> , le problème persiste
samedi 26/09, 20:50	Sortie de <code>beta-20260926-6</code> , toujours pas corrigé
dimanche 27/09, 00:04	Fin de l'export

Cause racine. Le rendu de l'overlay a été modifié dans `beta-20260924-3` et devient très lent quand un grand écran est combiné au bloom. J'ai écarté les autres pistes : sur ces rapports, le RTT reste autour de 35 ms et les métriques du serveur sont normales, donc ni le réseau ni le serveur ne sont en cause.

Impact. Sont touchés les joueurs équipés d'un écran de 2560 px ou plus avec le bloom activé, quel que soit le navigateur : 249 rapports sur 262, soit 44 joueurs. Les autres n'ont rien remarqué, ce qui explique que les plaintes ne venaient que d'une partie des joueurs.

Détection. Le problème n'a pas été repéré plus tôt parce qu'il n'existait aucune supervision, seulement des plaintes et un export manuel. La requête qui isole les écrans touchés :

```
sum by (width, bloom) (count_over_time(
  {service="game-telemetry", origin="export", event="perf_spike"}
  | json o="report.work.details.renderOverlay",
    width="report.graphics.width",
    bloom="report.graphics.bloom"
  | o > 100 [$_range]))
```

Actions. À court terme, revenir à `beta-20260922-1`, ou au minimum désactiver le bloom par défaut sur les grands écrans. Ensuite, corriger l'overlay et vérifier que sa part retombe sous 1 %. Pour éviter que ça se reproduise, je propose un déploiement progressif (canary) bloqué automatiquement si `RegressionOverlay` se déclenche, et un test de performance en 4K avec bloom avant chaque sortie.

Incident 2 : les abus

Farming. C'est la durée des parties qui m'a mis sur la piste. Une partie normale dure entre 3 et 14 minutes, alors que 83 parties ont été gagnées 3-0 sur vault en moins de 3 minutes. Elles sont concentrées sur deux nuits : le mercredi 23/09 de 01:14 à 03:46 (45 parties) et le vendredi 25/09 de 00:44 à 02:32 (38 parties). En temps normal, ce type de partie représente environ 3 % du total ; sur ces deux créneaux, on approche les 100 %. L'anti-triche n'en a mis que 17 en quarantaine sur 83.

Rapports falsifiés. Ils viennent tous du client `5e1f0c7a`. Entre le 24/09 à 09:35 et le 26/09 à 22:37, il a envoyé 70 rapports incohérents : des temps de rendu négatifs (jusqu'à -101 ms), et des valeurs entre 240 et 1 200 fps alors que ses frames durent environ 55 ms. Ces rapports faussent les statistiques, et il peut aussi s'agir de quelqu'un qui teste les limites de l'API.

```
sum by (client) (count_over_time(
  {service="game-telemetry", origin="export", event="perf_spike"}
  | json category | category="falsifie"
  | regexp `"id":\s*"P-(?P<client>[0-9a-f]{8})` [$_range]))
→ 5e1f0c7a : 70
```

Actions. Annuler les 83 victoires, examiner les comptes concernés et bloquer le client `5e1f0c7a`. Sur le long terme, le serveur devrait rejeter les rapports incohérents dès la réception, et l'anti-triche devrait prendre en compte la durée des parties et la répétition des scores sur une même carte. Les alertes FarmingSuspecte et RapportsFalsifies doivent rester actives.

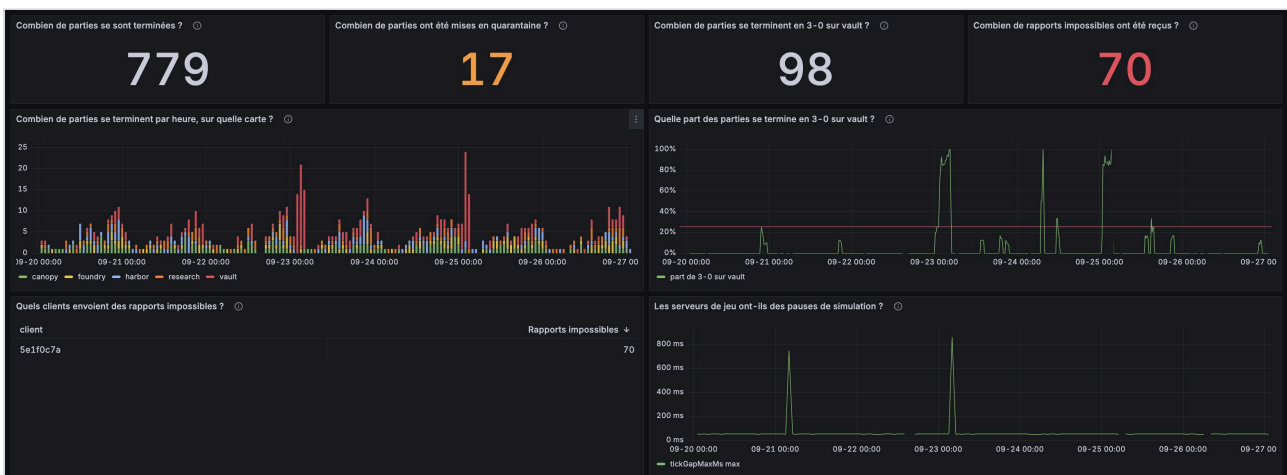


Figure 3 : le dashboard « Activité et intégrité », avec les 17 parties en quarantaine, les 70 rapports impossibles et les pics de farming.

7. Conclusion

Les deux questions de la direction ont une réponse, appuyée par des requêtes que n'importe qui peut relancer. Le gain le plus concret porte sur la détection : avec RegressionOverlay en place (fenêtre de 15 minutes, puis 5 minutes de confirmation), la régression aurait été signalée dès le jeudi soir, peu après le premier rapport overlay de 20:16, au lieu de plusieurs jours de plaintes.

Certaines limites restent. Les données perdues le 22/09 ne sont pas récupérables. L'export ne contient pas les identifiants de compte, donc je ne peux pas dire combien de personnes sont derrière le farming. Enfin, la rétention de Loki est de 30 jours : sans archivage ailleurs, l'export historique finira par être purgé.